



数据库系统课程实验报告

实验名称:	触发器
实验日期:	2025.12.2
实验地点:	文宣楼 B308
提交日期:	2025.12.4

学号:	37220232203780
姓名:	马鑫
专业年级:	2023 级数字媒体技术
学年学期:	第三学年第一学期

1.实验目的

- 理解 openGauss 触发器的作用和工作原理
 - AFTER/BEFORE 触发器
 - 行级(row)触发器和语句级(statement)触发器
- 熟练掌握 openGauss 触发器的设计方法
- 熟练掌握 openGauss 触发器的定义、查看、禁止、启用和删除操作

2.实验内容和步骤

(1) 创建部门表 dept(deptno, deptname), 其中,

- deptno 为部门号, 定长为 2 的字符型, 主码
- deptname 为部门名, 最大长度为 20 的变长字符型, 非空

步骤如下:

```
lab8=> CREATE TABLE dept
lab8-> (
lab8(> deptno CHAR(2) PRIMARY KEY,
lab8(> deptname VARCHAR(20) NOT NULL
lab8(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "dept_pkey" for table "dept"
CREATE TABLE
```

(2) 创建 Teacher 表: Teacher(ID, job, Sal, deptno), 其中,

- ID 为教工号, 定长为 5 的字符型, 主码
- JOB 为职称, 最大长度为 20 的变长字符型, 非空
- SAL 为工资, 长度为 7 的数字型, 其中保留两位小数
- deptno 为部门号, 定长为 2 的字符型, 外码, 引用 dept 表中的主码 deptno

步骤如下:

```

lab8=> CREATE TABLE Teacher
lab8-> (
lab8(> ID CHAR(5) PRIMARY KEY,
lab8(> job VARCHAR(20) NOT NULL,
lab8(> Sal NUMERIC(7,2),
lab8(> deptno CHAR(2),
lab8(> FOREIGN KEY(deptno) REFERENCES dept(deptno)
lab8(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "teacher_pkey" for table "teacher"
CREATE TABLE

```

(3) 为 dept 表增加实验数据: ('01', 'CS'), ('02', 'SW'), ('03', 'MA'); 为 Teacher 表增加实验数据: ('10001', '教授', 3800, '01'), ('10002', '教授', 4100, '02'), ('10003', '副教授', 3500, '01'), ('10004', '助理教授', 3000, '03')

步骤如下:

```

lab8=> INSERT INTO dept VALUES('01','CS'),('02','SW'),('03','MA');
INSERT 0 3

```

```

lab8-> INSERT INTO Teacher VALUES('10001','教授',3800,'01'),('10002','教授',4100,'02'),('10003','副教授',3500,'01'),('10004','助理教授',3000,'03');
INSERT 0 4

```

(4) 在 Teacher 表上创建一个 BEFORE 行级触发器 (名称: INSERT_OR_UPDATE_SAL) 以实现如下完整性规则: 教授的工资不得低于 4000 元, 如果低于 4000 元, 自动改为 4000 元

步骤如下:

```

lab8-> CREATE OR REPLACE FUNCTION insert_or_update_sal_func() RETURNS TRIGGER AS $$ DECLARE BEGIN if NEW.job='教授' AND NEW.Sal<4000.00 then New.Sal
:=4000.00; end if; RETURN NEW;END;$$ LANGUAGE PLPGSQL;
CREATE FUNCTION

```

```

lab8-> CREATE TRIGGER insert_or_update_sal BEFORE INSERT OR UPDATE ON Teacher FOR EACH ROW EXECUTE PROCEDURE insert_or_update_sal_func();
CREATE TRIGGER

```

(5) 验证触发器是否正常工作: 分别执行以下 A, B 两种操作, 验证 INSERT_OR_UPDATE_SAL 触发器是否被触发? 工作是否正确? 如果正确, 请观察 Teacher 表中数据的变化是否与预期一致

A. 插入两条新数据 ('10005', '教授', 3999, '02'), ('10006', '教授', 4000, '03')

B. 更新数据:将 id 为 10002 的教授工资改为 3900

步骤如下:

A1.插入数据正常

```
lab8=> INSERT INTO Teacher VALUES('10005','教授',3999,'02'),('10006','教授',4000,'03');  
INSERT 0 2
```

A2.查询, 发现 10005 教授的工资被修正为了 4000.00

```
lab8=> SELECT * FROM Teacher;  
  id  |   job   |   sal   | deptno  
-----+-----+-----+-----  
10001 | 教授    | 4000.00 | 01  
10002 | 教授    | 4000.00 | 02  
10003 | 副教授  | 4000.00 | 01  
10004 | 助理教授 | 4000.00 | 03  
10005 | 教授    | 4000.00 | 02  
10006 | 教授    | 4000.00 | 03  
(6 rows)
```

B1.更新数据成功

```
lab8=> UPDATE Teacher SET Sal=3900.00 WHERE ID='10002';  
UPDATE 1
```

B2.查询, 发现 10002 教授的工资被修正

```
lab8=> SELECT * FROM Teacher;  
  id  |   job   |   sal   | deptno  
-----+-----+-----+-----  
10001 | 教授    | 4000.00 | 01  
10003 | 副教授  | 4000.00 | 01  
10004 | 助理教授 | 4000.00 | 03  
10005 | 教授    | 4000.00 | 02  
10006 | 教授    | 4000.00 | 03  
10002 | 教授    | 3900.00 | 02  
(6 rows)
```

(6) 查看触发器（名称和代码）

步骤如下：

```
lab8=> \ds Teacher;
          Table "lab8.teacher"
  Column |          Type          | Modifiers
-----+-----+-----
 id      | character(5)            | not null
 job     | character varying(20)   | not null
 sal     | numeric(7,2)            |
 deptno  | character(2)            |
Indexes:
    "teacher_pkey" PRIMARY KEY, btree (id) TABLESPACE pg_default
Foreign-key constraints:
    "teacher_deptno_fkey" FOREIGN KEY (deptno) REFERENCES dept(deptno)
Triggers:
    insert_or_update_sal BEFORE INSERT OR UPDATE ON teacher FOR EACH ROW EXECUTE PROCEDURE insert_or_update_sal_func()

lab8=> SELECT pg_get_triggerdef(oid) FROM pg_trigger WHERE tgname='insert_or_update_sal';
          pg_get_triggerdef
-----
CREATE TRIGGER insert_or_update_sal BEFORE INSERT OR UPDATE ON teacher FOR EACH ROW EXECUTE PROCEDURE insert_or_update_sal_func()
(1 row)
```

(7)设计触发器自动维持表间的外码约束:删除 dept 表中 deptno 为 03 的数据后，teacher 表上引用该数据的记录也被自动删除。思考并验证：如何实现在删除 dept 表中 deptno 为 01 的数据后自动将 teacher 表中引用了该值（即 01）的 deptno 字段值设置为 null。

步骤如下：

1.创建触发器

```
lab8=> CREATE OR REPLACE FUNCTION cascade_delete_func() RETURNS TRIGGER AS $$ DECLARE BEGIN if OLD.deptno='01' then UPDATE Teacher SET deptno=NULL W
HERE deptno=OLD.deptno; else if OLD.deptno='03' then DELETE FROM Teacher WHERE deptno=OLD.deptno; end if; end if; RETURN OLD;END;$$ LANGUAGE PLPGSQL
;
CREATE FUNCTION

lab8=> CREATE TRIGGER cascade_delete BEFORE DELETE ON dept FOR EACH ROW EXECUTE PROCEDURE cascade_delete_func();
CREATE TRIGGER
```

2.进行删除验证

```
lab8=> DELETE FROM dept WHERE deptno='03';
DELETE 1
```

```
lab8=> SELECT * FROM Teacher;
```

id	job	sal	deptno
10001	教授	4000.00	01
10003	副教授	4000.00	01
10005	教授	4000.00	02
10002	教授	4000.00	02

(4 rows)

```
lab8=> DELETE FROM dept WHERE deptno='01';
DELETE 1
```

```
lab8=> SELECT * FROM Teacher;
```

id	job	sal	deptno
10005	教授	4000.00	02
10002	教授	4000.00	02
10001	教授	4000.00	
10003	副教授	4000.00	

(4 rows)

(8) 现有选修表 SC(Sno,Cno,grade),各字段含义分别为学号（数据类型：定长为 9 的字符型），课程号（数据类型：定长为 3 的字符型）和课程成绩（数据类型：长度为 3 的整型），样本数据集为 {(202315121,1,92),(202315121,2,85),(202315121,3,88),(202315122,2,90),(202315122,3,80)}。要求设计一个触发器实现审计日志记录：当对表 SC 的 Grade 属性进行修改时，若分数增加了 10%及其以上，则将此次操作记录到下面表中：SC_U(Sno, Cno, Oldgrade, Newgrade)，其中各字段含义分别为学号(同 SC.SNO)，课程号(同 SC.CNO)，修改前的分数和修改后的分数(数据类型同 sc.grade)。

① 创建 SC_U 表: SC_U(Sno, Cno, Oldgrade, Newgrade)

② 创建 SC 表上的 AFTER 行级触发器, 触发器名为 tri_update_sc

③ 验证 tri_update_sc 触发器是否正常工作。

UPDATE SC

SET grade=100

WHERE Sno=' 202315122' AND Cno=' 2' ;

UPDATE SC

SET grade=90

WHERE Sno=' 202315121' AND Cno=' 2' ;

要求: 执行上述两种操作, 如果触发器正确工作, 请观察 SC_U 表中数据的变化。

步骤如下:

1.创建 SC 表并填入数据

```
lab8=> CREATE TABLE SC
lab8-> (
lab8(> Sno CHAR(9),
lab8(> Cno CHAR(3),
lab8(> grade INT CHECK (grade>=0 AND grade <=100),
lab8(> PRIMARY KEY(Sno,Cno)
lab8(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "sc_pkey" for table "sc"
CREATE TABLE

lab8=> INSERT INTO SC VALUES('202315121','1',92),('202315121','2',85),('202315121','3',88),('202315122','2',90),('202315122','3',80);
INSERT 0 5
```

2.创建 SC_U 表

```

lab8=> CREATE TABLE SC_U
lab8-> (
lab8(> log_id SERIAL PRIMARY KEY,
lab8(> Sno CHAR(9),
lab8(> Cno CHAR(3),
lab8(> Oldgrade INT,
lab8(> Newgrade INT,
lab8(> change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
lab8(> change_percent NUMERIC(5,2),
lab8(> CHECK (Oldgrade IS NOT NULL AND Newgrade IS NOT NULL)
lab8(> );
NOTICE: CREATE TABLE will create implicit sequence "sc_u_log_id_seq" for serial column "sc_u.log_id"
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "sc_u_pkey" for table "sc_u"
CREATE TABLE

```

3.创建触发器

```

lab8=> CREATE OR REPLACE FUNCTION audit_grade_change() RETURNS TRIGGER AS $$ DECLARE percent_change NUMERIC(5,2); BEGIN percent_change:=((NEW.grade-OLD.grade)*100.0)/OLD.grade; if percent_change>=10.0 then INSERT INTO SC_U(Sno,cno,oldgrade,newgrade,change_percent) VALUES(OLD.Sno,OLD.Cno,OLD.grade,NEW.grade,percent_change); end if; RETURN NEW; END; $$ LANGUAGE PLPGSQL;
CREATE FUNCTION

```

```

lab8=> CREATE TRIGGER tri_update_sc AFTER UPDATE OF grade ON SC FOR EACH ROW WHEN (OLD.grade IS DISTINCT FROM NEW.grade) EXECUTE PROCEDURE audit_grade_change();
CREATE TRIGGER

```

4.验证

```

lab8=> UPDATE SC SET grade=100 WHERE Sno='202315122' AND Cno='2';
UPDATE 1
lab8=> UPDATE SC SET grade=90 WHERE Sno='202315121' AND Cno='2';
UPDATE 1

```

```

lab8=> SELECT * FROM SC_U;
log_id | sno | cno | oldgrade | newgrade | change_time | change_percent
-----+-----+-----+-----+-----+-----+-----
1 | 202315122 | 2 | 90 | 100 | 2025-12-04 16:39:34.416981 | 11.11
(1 row)

```

(9) 将触发器 tri_update_sc 改名为 update_sc_tri

步骤如下：

```

lab8=> ALTER TRIGGER tri_update_sc ON SC RENAME TO update_sc_tri;
ALTER TRIGGER

```

(10) 验证触发器禁用后效果

- ① 将数据还原到步骤(5)之前，即触发器工作前的原数据
- ② 修改 SC 表使 AFTER_UPDATE_SC 触发器失效
- ③ 再次执行上面的步骤(8)之③，验证触发器被禁用后是否还能正常工作

步骤如下：

1.还原数据

```
lab8=> UPDATE SC SET grade=90 WHERE Sno='202315122' AND Cno='2';  
UPDATE 1  
lab8=> UPDATE SC SET grade=85 WHERE Sno='202315121' AND Cno='2';  
UPDATE 1
```

```
lab8=> SELECT * FROM SC;  
      sno      | cno | grade  
-----+-----+-----  
202315121 | 1   | 92  
202315121 | 3   | 88  
202315122 | 3   | 80  
202315122 | 2   | 90  
202315121 | 2   | 85  
(5 rows)
```

```
lab8=> DELETE FROM SC_U;  
DELETE 1
```

```
lab8=> SELECT * FROM SC_U;  
log_id | sno | cno | oldgrade | newgrade | change_time | change_percent  
-----+-----+-----+-----+-----+-----+-----  
(0 rows)
```

2.使触发器失效

```
lab8=> ALTER TABLE SC DISABLE TRIGGER update_sc_tri;  
ALTER TABLE
```

3.再次执行步骤（8）之③，触发器没有执行

```
lab8=> UPDATE SC SET grade=100 WHERE Sno='202315122' AND Cno='2';  
UPDATE 1  
lab8=> UPDATE SC SET grade=90 WHERE Sno='202315121' AND Cno='2';  
UPDATE 1
```

```
lab8=> SELECT * FROM SC_U;  
log_id | sno | cno | oldgrade | newgrade | change_time | change_percent  
-----+-----+-----+-----+-----+-----+-----  
(0 rows)
```

(11) 删除所创建的触发器

步骤如下：

```
lab8=> DROP TRIGGER update_sc_tri ON SC;  
DROP TRIGGER
```

```
lab8=> DROP TRIGGER insert_or_update_sal ON Teacher;  
DROP TRIGGER
```

```
lab8=> DROP TRIGGER cascade_delete ON dept;  
DROP TRIGGER
```

3.实验总结

3.1 完成的工作

完成实验内容，成功创建，测试，删除触发器。

3.2 对实验的认识

通过本实验更加深入理解了触发器在数据库中的应用，并了解到 OpenGauss 触发器语法与课本的区别。

思考题：

1.简述 openGauss 触发器的作用及适用场景。

答：openGauss 能够使得在特定的数据库事件发生在之前或之后自动执行一段预定义的 PL/pgSQL 代码，从而能够确保数据的一致性、实现复杂审计、自动派生与同步、实施复杂约束等。适用于需要自动化、强制化执行业务逻辑的场景比如审计与日志记录、维护衍生数据与同步、实施超出标准约束的复杂业务规则、数据清洗与格式化等。

2.现有一个 oracle 触发器（代码如下），请编写一个实现相同功能的 openGauss 触发器。

```
CREATE OR REPLACE TRIGGER dept_update
```

```

AFTER update on dept
FOR EACH ROW
BEGIN
    update emp
    set deptno=:new.deptno
    where deptno=:old.deptno;
END;

```

答：CREATE OR REPLACE FUNCTION dept_update_func()

```

RETURNS TRIGGER AS $$
BEGIN
    update emp set deptno=NEW.deptno
    where deptno=OLD.deptno;

    RETURN NEW;
END;

$$ LANGUAGE PLPGSQL;

CREATE TRIGGER dept_update
    AFTER UPDATE ON dept
    FOR EACH ROW
    EXECUTE FUNCTION dept_update_func();

```

3.3 遇到的困难及解决方法

无。